Spring Security and JWT

8 min read Updated 28 Jul 2026 Topic 61 of 65

Why it matters

Security is where a plausible-sounding answer can be dangerously wrong. "We store the JWT in localStorage" and "we can revoke a token by deleting it" both sound fine and are both mistakes. Interviewers probe here because the consequences are real, and because knowing *why* stateless tokens are hard to revoke shows you understand the mechanism rather than the library.

The concept, simply

Spring Security is a **chain of servlet filters** that runs before the `DispatcherServlet` (Chapter 11). Each filter has one job — read a token, authenticate, check authorisation — and the request only reaches your controller if it survives.

Two distinct steps, routinely conflated: **authentication** is "who are you?"; **authorisation** is "may you do this?".

A **JWT** is three base64url segments — header, payload, signature — joined by dots. The signature proves the token was issued by someone holding the key and has not been altered.

TEXT Copy

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhc2hhIiwiZXhwIjoxNzM1Njg5NjAwfQ.k3f9 ...

// ⚠ The payload is base64, NOT encrypted. Anyone holding the token can read
```

// it — paste it into jwt.io and every claim is visible. Never put anything
// confidential in a JWT: no PII beyond an id, no internal flags, no secret

How it works

A typical stateless flow: the client posts credentials, the server verifies them and returns a signed JWT, and the client sends it as `Authorization: Bearer <token>` on every subsequent request. A filter validates the signature and expiry and populates the `SecurityContext`. No session is stored — which is the selling point and the entire problem.

The revocation problem is the crux. A stateless token is valid until it expires because the server holds no record of it. Log a user out, disable an account, or discover a stolen token, and the token keeps working. The three answers:

- **Short-lived access tokens** (5–15 minutes) plus a long-lived refresh token that *is* stored server-side and can be revoked. The standard design.
- A **denylist** of revoked token ids in Redis — which reintroduces server state, i.e. gives up the property you chose JWT for.
- **Sessions instead**, which are trivially revocable. For a single application with no cross-service needs, a plain session cookie is often the better engineering choice — a genuinely good answer, because it questions the premise.

Code examples

JAVA

Copy

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain chain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable())          // ✓ ok for a stateless
                                                   // ✗ NEVER for cookie s
```

```
                .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())      // ✓ deny by default, l
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilt
            .build();
    }

    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();         // ✓ adaptive + salted
    }
}
// ⚠ Rule ORDER matters — the first match wins. Putting .anyRequest()
//   .permitAll() before a specific rule silently opens everything.
```

Password storage, which is asked on its own:

JAVA Copy

```
// ✗ Never: plain text; MD5 or SHA-256 (fast, so brute-forceable); a share
// ✓ BCrypt, scrypt or Argon2 — deliberately SLOW and per-password salted.
String hash = encoder.encode(rawPassword);    // salt is embedded in the ou
boolean ok = encoder.matches(rawPassword, hash);

// The point of an adaptive hash is that it can be made slower as hardware
// improves: BCrypt's strength parameter raises the cost, so a stolen
// database stays expensive to attack years later. A general-purpose hash
// like SHA-256 is fast BY DESIGN, which is exactly wrong here.
```

Method-level authorisation, for rules the URL cannot express:

JAVA Copy

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteAll() { … }

// Ownership checks — the URL alone cannot express "your own order":
@PreAuthorize("@orderSecurity.isOwner(#orderId, authentication.name)")
```

```
public Order get(@PathVariable Long orderId) { … }

// ⚠ These are AOP-based, so Chapter 11's proxy rules apply: a self-invoca
//   bypasses the check entirely. A security check that silently does not
//   is considerably worse than a missing transaction.
```

Under the hood

Where to store a token on the client is the question with the sharpest trade-off. `localStorage` is readable by any JavaScript on the page, so a single XSS flaw hands over every user's token. An `HttpOnly`, `Secure`, `SameSite` cookie cannot be read by JavaScript at all — but cookies are sent automatically, which reintroduces CSRF. The usual recommendation is the cookie plus CSRF protection, because XSS is both more common and more catastrophic. Saying "localStorage, it is simpler" without naming the XSS exposure is a red flag.

CSRF protection depends on the auth mechanism, which is why the blanket advice to disable it is wrong. If credentials travel in a cookie the browser attaches automatically, you need CSRF protection. If they travel in an `Authorization` header that the browser never adds by itself, CSRF does not apply and disabling it is correct. Disabling it "because the tutorial did" while using cookie sessions is a genuine vulnerability.

Algorithm confusion is worth naming: a classic JWT attack sets `alg` to `none`, or swaps RS256 for HS256 so the public key is used as an HMAC secret. Any library configured to trust the token's own `alg` header is vulnerable. Always pin the expected algorithm server-side rather than reading it from the token you are validating.

Common mistakes

- **Putting sensitive data in a JWT.** The payload is base64, not encrypted.
- **Long-lived access tokens.** They cannot be revoked — keep them to minutes and use refresh tokens.
- **Storing tokens in `localStorage`.** One XSS flaw exposes every token.
- **Disabling CSRF with cookie-based auth.** A real vulnerability, not a formality.
- **Fast hashes for passwords.** SHA-256 is fast by design; use BCrypt, scrypt or Argon2.

- **Self-invoking a `@PreAuthorize` method.** The proxy is bypassed and the check never runs.

Interview lens

Q: "How does Spring Security work?"

A: As a chain of servlet filters registered ahead of the `DispatcherServlet`. Each filter has a narrow job — extract credentials, authenticate, populate the `SecurityContext`, enforce authorisation — and a request only reaches a controller if it passes. Because it is filter-based, security rejections happen before MVC, which is why `@ControllerAdvice` never sees a 401.

Q: "How do you revoke a JWT?"

A: You largely cannot, and that is the fundamental trade-off — the server keeps no record, so the token is valid until it expires. The standard mitigation is short-lived access tokens, five to fifteen minutes, with a long-lived refresh token stored server-side that can be revoked. A denylist in Redis works but reintroduces the server state you chose JWT to avoid. For a single application with no cross-service requirement, ordinary sessions are often the better choice precisely because revocation is trivial.

Q: "Is a JWT encrypted?"

A: No — signed, not encrypted. The payload is base64url and anyone holding the token can read every claim. The signature only proves integrity and origin. So a JWT may carry a user id and roles, but never passwords, personal data or internal secrets. JWE exists for encryption, but it is rarely what people mean by "JWT".

A: Preferably an `HttpOnly`, `Secure`, `SameSite` cookie, which JavaScript cannot read, so an XSS flaw cannot steal it — with CSRF protection, since cookies are sent automatically. `localStorage` is simpler and means one XSS bug leaks every user's token. The trade is XSS exposure versus CSRF exposure, and XSS is the more damaging of the two.

Q: "How do you store passwords?"
A: With an adaptive hash — BCrypt, scrypt or Argon2 — which is deliberately slow and salts each password individually. The salt is embedded in the output, so there is nothing separate to manage. General-purpose hashes like SHA-256 are wrong here because they are fast by design, which is exactly what an attacker with a stolen database wants. Adaptive hashes also have a cost parameter you can raise as hardware improves.

Q: "When can you disable CSRF protection?"
A: When authentication does not rely on anything the browser attaches automatically — a stateless API using an `Authorization` header, for instance, since an attacker's site cannot make the browser add that header. If credentials are in a cookie, CSRF protection is required. Disabling it by default because a tutorial did is one of the more common real vulnerabilities in Spring applications.

Cheat-card

- Security is a **filter chain before MVC** → `@ControllerAdvice` never sees 401/403.
- Authentication = who; authorisation = may you.
- **A JWT is signed, not encrypted** — never put secrets or PII in it.
- **Cannot be revoked** → short-lived access + revocable refresh token (or just use sessions).
- Store in an `HttpOnly` / `Secure` / `SameSite` **cookie + CSRF**, not `localStorage`.

- **Pin the JWT algorithm** server-side — never trust `alg`.
- Rule order matters (first match wins); `anyRequest().authenticated()` last.
- `@PreAuthorize` is proxy-based — **self-invocation skips the check**.

QUICK CHECK — no marks, just for you.

1. Is a JWT encrypted?

- ◯ Only when transmitted over HTTPS
- ◯ No — it is SIGNED, not encrypted. The payload is base64url and anyone holding the token can read every claim
- ◯ Yes, with the server's secret key
- ◯ Yes, but only the payload section

Check answer

2. How do you revoke a JWT before it expires?

- ◯ Delete it from the server's token table
- ◯ Call the standard JWT revoke endpoint
- ◯ Largely you cannot — the server keeps no record, so the mitigations are short-lived access tokens with a revocable refresh token, a denylist that reintroduces server state, or using sessions instead
- ◯ Change the user's password, which invalidates all their tokens

Check answer

- Always for REST APIs
- Never
- Only in development
- When authentication does not rely on anything the browser attaches automatically — for example an Authorization header, which an attacker's site cannot make the browser add

Check answer

Previous
REST API Design in Spring

Next
Testing: JUnit 5, Mockito and Spring Slices

LEARN MORE

ARTICLES & DOCS

- Spring Security reference
  docs.spring.io
- The security filter chain
  docs.spring.io
- JSON Web Token — RFC 7519
  www.rfc-editor.org

VIDEOS



Spring Boot 3 + Spring Security 6 - JWT Authentication and Authorisation [NEW] [2023]
Amigoscode

Hand-picked references for this lesson.